

GitHub Pages + WebRTC 近接対戦 通信構成ガイド

サーバーなしで「近くにいる人のスマホ同士」をつなぐ Web ゲームの作り方 一トークセロで実装・検証した構成を、他のゲーム／リポジトリに流用できる形でまとめたもの

2026年9月27日・参照実装: github.com/Muaaaaa849/othelo ([src/lib/p2p/](#), [/link/](#) ページ)

要点(1分で)

- アプリは静的サイトとして GitHub Pages で配信 (HTTPS・無料)。ゲームのサーバーは持たない。
- スマホ同士は WebRTC DataChannel で直接通信。つなぐための情報 (SDP) は圧縮して QRコードを2回読み合うだけで交換する (シグナリングサーバー不要)。
- Bluetooth は Web では使えない (Web Bluetooth は接続しに行く側のみ・iPhone非対応)。
- 同じ Wi-Fi かテザリングなら確実。別回線どうしは STUN でつながることが多いが保証はない (確実にするなら TURN)。
- 流用に必要なのは TypeScript 2ファイル (約180行) + npm 2つ (`qrcode`, `jsqr`)。フレームワーク非依存。

1. 全体構成
2. なぜこの構成か (方式の比較)
3. 接続の流れ
4. 接続コードの形式
5. ネットワーク条件とつながりやすさ
6. 別リポジトリへの流用手順
7. API リファレンス
8. ゲーム通信の設計パターン
9. 検証結果
10. 制約・拡張のしかた
11. 付録 (全ソース・ワークフロー)

1. 全体構成

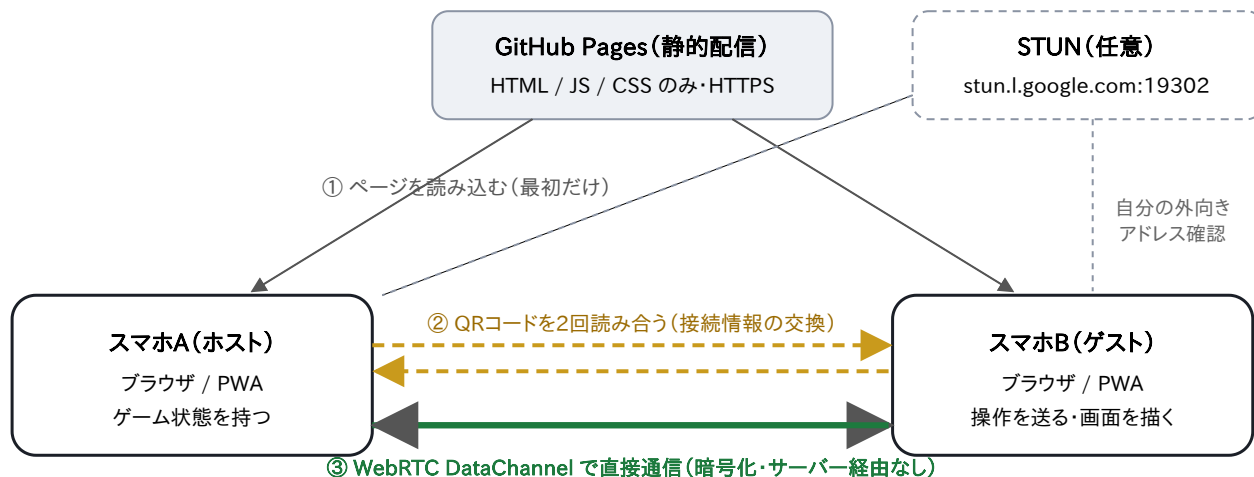


図1 構成要素。ゲーム中にサーバーが関わるのは、STUN による自分のアドレス確認 (任意) だけ。

要素	役割	この構成での実体
静的ホスティング	アプリ本体 (HTML/JS) を配る。HTTPS 必須 (カメラ・一部APIのため)。	GitHub Pages + GitHub Actions (自動公開)
シグナリング	WebRTC 接続前に、互いの接続情報 (SDP・ICE候補) を渡す。	QRコード (カメラで読む) / 予備にテキストのコピー
NAT 越え補助	別回線どうしの際、外から見た自分のアドレスを知る。	公開STUN (無料)。同じWi-Fiなら不要
データ通信	ゲームの操作・状態を送る。順序保証・再送あり・暗号化 (DTLS)。	RTCDataChannel (ordered: true)、JSON

2. なぜこの構成か (方式の比較)

方式	Android	iPhone	サーバー	評価
WebRTC + QRシグナリング (本書)	○	○	不要 (STUN のみ任意)	採用。静的サイトのまま動く
Web Bluetooth	接続しに行く側のみのみ	非対応	不要	スマホ同士は不可 (どちらも待ち受け・広告ができない)
WebRTC + シグナリングサーバー (PeerJS 公開サーバー、Firebase 等)	○	○	必要 (無料枠あり)	部屋コード入力だけで楽。外部サービスに依存
WebSocket でサーバー中継 (Firebase / Supabase / PartyKit 等)	○	○	必要	遠隔対戦も確実。常時サーバー経由・アカウント管理が必要
ネイティブアプリ (Nearby Connections / MultipeerConnectivity)	○	○	不要	Bluetooth 可。ただし Android⇄iPhone 間は困難・ストア配布が必要
Web NFC	一部	非対応	不要	数百バイトを一瞬渡すだけ。対戦通信には使えない

3. 接続の流れ



図2 接続シーケンス。ICE候補は trickle せず、全部そろえてから1回のコードで渡す(QRを2回読むだけで済む)。

- ・ **ホスト=最初にQRを見せる側、ゲスト=読む側。**ゲームの親(状態を持つ側)もホストにすると設計が単純になる(8章)。
- ・ 候補収集の待ちは最大4秒。STUN に届かない環境では毎回4秒待つ(同じWi-Fiではそれでもつながる)。
- ・ QRが読めない時のために、同じ文字列をテキストでコピー／貼り付けできる逃げ道を必ず用意する。

4. 接続コードの形式

項目	内容
形式	"TS1." + base64url(deflate-raw(JSON { t: "o" "a", s: SDP })) (TS1 はバージョン。互換性のない変更時は TS2 に上げる)
SDP の軽量化	DataChannel だけの接続に不要な a=extmap-allow-mixed / a=msid-semantic / a=max-message-size 行を削除
圧縮	ブラウザ標準の CompressionStream("deflate-raw") (Chrome 80+ / Safari 16.4+ / Node 18+)。ライブラリ不要
サイズ(実測)	offer 約530文字・answer 約527文字 → QR(誤り訂正L)でバージョン20前後。画面幅いっぱいに表示すれば読める
QR 表示／読取	表示 qrcode (toCanvas, errorCorrectionLevel "L", margin 2)／読取 jsqr (getUserMedia の背面カメラ映像を150msごとに解析)
安全性	コードには接続用の一時的な鍵の指紋・ローカルアドレス(mDNSで匿名化済み)が入る。通信本体は DTLS で暗号化。コードを第三者に見せない運用で十分

さらに小さくしたい場合は、SDP から必要な値(ufrag, pwd, fingerprint, 候補)だけを取り出して受信側でSDPを組み立て直す方式で、100バイト前後まで縮められる(参考: QWBP)。ただし実装が複雑になるため、本構成では「圧縮のみ」を採用した。

5. ネットワーク条件とつながりやすさ

状況	見込み	理由・対策
2台とも同じ家庭用 Wi-Fi	◎	LAN内で直接(host候補)。インターネットが不安定でも通信自体は LAN 内で完結
片方のスマホのテザリングに、もう片方が接続	○	同じLANになる。Wi-Fiがない場所での推奨手段(実機での確認を推奨)
別々のモバイル回線／別々のWi-Fi	△	STUN で外向きアドレスを交換してつながることが多い。キャリアの NAT方式によっては失敗 → TURN(中継)が必要
公衆Wi-Fi・ホテル・会社のゲストWi-Fi	×になりやすい	端末どうしの通信を禁止(APアイソレーション)していることが多い → テザリングに切り替える
VPN・一部のセキュリティアプリ使用中	△	UDP が遮断されるとつながらない。オフにして再試行

ページ自体は最初にインターネットから読み込む必要がある(Service Worker によるオフライン化は未実装)。一度開いた後の通信は、同じLAN内なら外部に出ない。

6. 別リポジトリへの流用手順

6.1 コピーするもの

ファイル(付録)	内容	依存
p2p/signal.ts (付録A)	接続コードのエンコード／デコード	なし
p2p/peer.ts (付録B)	WebRTC 接続・送受信・経路判定(Peer クラス)	signal.ts
QrCode.tsx / QrScanner.tsx (付録C)	React 用の QR 表示・カメラ読取(React 以外は 7.3 のバニラ版)	qrcode, jsqr

```
npm install qrcode jsqr
npm install -D @types/qrcode # TypeScript の場合
```

6.2 GitHub Pages で配信する

- ビルド結果が静的ファイルになる構成にする(Vite ならそのまま `dist/`、Next.js は `output: "export"`)。
- リポジトリ名のサブパス(`https://ユーザー.github.io/リポジトリ/`)で配信されるので、ベースパスを設定する(Vite: `base: "/リポジトリ/"`、Next.js: `basePath`。本構成では環境変数 `NEXT_PUBLIC_BASE_PATH` で渡す)。manifest の `start_url`・アイコンのパスにも付ける。
- `.github/workflows/deploy-pages.yml` (付録D)を置き、ビルド出力フォルダ(`out / dist`)と対象ブランチを合わせる。
- GitHub の **Settings** → **Pages** → **Source** を「GitHub Actions」にする(最初の1回だけ・管理者のみ)。未設定だと `deploy` が `404 Ensure GitHub Pages has been enabled` で失敗する。
- 公開リポジトリなら無料。非公開リポジトリの Pages は有料プランが必要(その場合は Vercel / Netlify / Cloudflare Pages が無料で代替可)。

6.3 チェックリスト

- HTTPS で配信している(カメラ `getUserMedia` と `CompressionStream` のため。localhost は例外的に可)

- `<video playsInline muted>` を付けている (iPhone でインライン再生させるため)
- カメラ拒否時・QR が読めない時のテキスト入力の逃げ道がある
- 画面遷移しても `Peer` インスタンスが破棄されない場所 (アプリ全体の状態・Context 等) に置いている
- 切断 (`onClose`) 時の表示と「最初からやり直す」導線がある

7. API リファレンス

7.1 Peer クラス (付録B)

メソッド	説明
<code>new Peer({ onOpen, onMessage, onClose }, iceServers?)</code>	接続を1本作る。 <code>onMessage</code> には <code>JSON.parse</code> 済みの値が来る。 <code>iceServers</code> 省略時は Google の公開STUN
<code>createOffer(): Promise<string></code>	ホスト用。 <code>DataChannel</code> を作り、接続コード①を返す
<code>answerOffer(code): Promise<string></code>	ゲスト用。①を受け取り、返事の接続コード②を返す
<code>acceptAnswer(code): Promise<void></code>	ホスト用。②を受け取る。成功すると両方で <code>onOpen</code>
<code>send(data): boolean</code>	JSON にして送る。未接続なら <code>false</code>
<code>route(): Promise<"LAN" "INTERNET" "RELAY" "UNKNOWN"></code>	実際に使われている経路 (<code>getStats</code> の選択中候補ペアから判定)
<code>close()</code>	切断

7.2 使い方

```
import { Peer } from "../p2p/peer";          // 付録A・Bのファイル
import QRCode from "qrcode";
import jsQR from "jsqr";

// ---- ホスト ----
const host = new Peer({
  onOpen: () => console.log("つながった"),
  onMessage: (msg) => console.log("受信", msg),
  onClose: () => console.log("切れた"),
});
const offer = await host.createOffer();        // "TS1.xxxx" (約530文字)
await QRCode.toCanvas(document.querySelector("canvas"), offer, { errorCorrectionLevel: "L", margin: 2 });
// ...ゲストの返事QRをカメラで読んだら:
await host.acceptAnswer(answerText);

// ---- ゲスト ----
const guest = new Peer({ onOpen, onMessage, onClose });
const answer = await guest.answerOffer(offerText); // ホストのQRを読んだ文字列
await QRCode.toCanvas(canvas, answer, { errorCorrectionLevel: "L", margin: 2 });

// ---- つながった後 ----
host.send({ t: "move", seq: 1, row: 2, col: 3 }); // JSON で送受信
console.log(await host.route());                  // "LAN" | "INTERNET" | "RELAY"
```

7.3 フレームワークなしのQR読み取り

```
// フレームワークなしのカメラ読み取り (React版は付録C)
async function scanQr(video, onResult) {
  const stream = await navigator.mediaDevices.getUserMedia({
    video: { facingMode: "environment", width: { ideal: 1280 }, height: { ideal: 720 }, audio: false });
  video.srcObject = stream; video.playsInline = true; video.muted = true; await video.play();
  const c = document.createElement("canvas"), ctx = c.getContext("2d", { willReadFrequently: true });
  const timer = setInterval(() => {
```

```

    if (!video.videoWidth) return;
    c.width = video.videoWidth; c.height = video.videoHeight; ctx.drawImage(video, 0, 0);
    const img = ctx.getImageData(0, 0, c.width, c.height);
    const r = jsQR(img.data, img.width, img.height, { inversionAttempts: "dontInvert" });
    if (r?.data) { clearInterval(timer); stream.getTracks().forEach(t => t.stop()); setResult(r.data); }
  }, 150);
}

```

8. ゲーム通信の設計パターン

- **ホスト権威型**にする: ゲーム状態はホストだけが持つ。ゲストは「ここに置きたい」等の意図だけ送り、ホストが合法性を判定して**状態まるごと**を返す。ずれ・二重操作が起きない。ターン制ゲームなら状態は数KB以下で十分速い(実測往復1~2ms、LAN)。
- ゲームロジックは UI から分離した純粋関数 (`reduce(state, action)`) にし、ホスト・ゲストで同じコードを使う(トークセロの `src/engine` と同じ考え方)。
- メッセージには種類 `t`・連番 `seq`・接続直後に**プロトコル版 v** を入れる。版が違えば「アプリを再読み込みしてね」と表示する。
- `DataChannel` は `ordered: true` (既定で再送あり)なので、ターン制なら到着順・欠落を自分で扱う必要はない。
- 乱数(シャッフル等)はホストだけが引き、結果を状態として配る。
- 3人以上: ホストが各ゲストと1本ずつ接続するスター型にする(ゲストごとにQR交換が必要)。

```

// ホスト権威型: 状態はホストだけが持ち、ゲストは「やりたいこと」だけ送る
type ToHost = { t: "hello"; v: 1; name: string } // 接続直後にバージョン確認
              | { t: "intent"; seq: number; action: Action }; // 例: { kind: "place", row, col }
type ToGuest = { t: "welcome"; v: 1; you: "WHITE" }
                | { t: "state"; seq: number; state: GameState } // 毎回まるごと送る(数KB以下なら十分速い)
                | { t: "reject"; seq: number; reason: string };

// ホスト側
peer = new Peer({ onMessage: (m) => {
  if (m.t === "intent" && isLegal(state, "WHITE", m.action)) {
    state = reduce(state, m.action); // ゲームロジックは両端で同じものを使う
    render(state); peer.send({ t: "state", seq: m.seq, state });
  } else if (m.t === "intent") peer.send({ t: "reject", seq: m.seq, reason: "illegal" });
}});

```

9. 検証結果(2026年9月27日)

検証	環境	結果
接続コード エンコード／デコード・破損検出	Vitest (Node 22)	3件合格
テキストコードで接続・双方向送受信	別プロセスの Chromium 2つ (Pixel 7 / iPhone 14 のエミュレーション)	接続・5回と3回の送信がすべて到達・往復1〜2ms・経路 LAN
カメラでQRを読んで接続(擬似カメラに相手画面のQRを映す)	同上 (Chromium の擬似カメラ機能)	読取→返事作成 約1.0秒、読取→接続完了 約0.9秒
公開URL (GitHub Pages・HTTPS) で同じ試験	同上	接続・送受信OK
GitHub Actions でビルド→Pages 公開	GitHub	成功
実機 (Android⇄iPhone)・実カメラでの読みやすさ	—	未検証(要実機確認)
別回線 (STUN経由)・テザリング	—	未検証(検証環境から STUN に到達できず)

実機での確認手順: 2台で <https://muaaaa849.github.io/othelo/link/> を開く → 片方「ホストになる」、もう片方「ゲストになる」→ QRを読み合う → 表示される「経路」と「相手の画面を光らせる」で確認。

10. 制約・拡張のしかた

課題	対応
別回線でつながらない	TURN サーバーを <code>iceServers</code> に追加 (Cloudflare Calls / Twilio / Metered 等。多くは有料・認証情報の発行が必要)。静的サイトに固定の認証情報を置くと悪用されるので、短期トークンを発行する仕組みが要る
QRを2回読むのが面倒	シグナリングだけ外部に任せる (PeerJS 公開サーバー・Firebase Realtime Database 等) → 4〜6桁の部屋コード入力で接続。代わりに外部サービス依存になる
途中で切れた	最小構成では再接続なし。状態をホストが保持しているので、QRを読み直して新しい接続を作り、ホストが最新状態を送り直せば再開できる
QRが細かく読みにくい	画面を明るく・画面幅いっぱいに表示。改善するなら SDP を必要項目だけに絞る方式 (4章末)
オフラインで起動したい	Service Worker でアプリをキャッシュ (PWA化)。フォントも同梱する
Bluetooth で近くの人を自動検出したい	Web では不可。ネイティブアプリ化 (Capacitor 等+各OSの近距離通信API) が必要

参考

- MDN: Web Bluetooth API / WebBluetoothCG implementation-status.md (ブラウザは GATT クライアントのみ・広告/ペリフェラル非対応)
- WebKit Bug 101034 (Safari は Web Bluetooth 非対応) / caniuse: Web Bluetooth
- magarcia/qwbp: Serverless WebRTC signaling via QR codes / franklita.com: Serverless WebRTC using QR codes

付録A src/lib/p2p/signal.ts

```
/**
 * WebRTC の接続情報 (SDP) を QR コード / テキストで受け渡すための圧縮エンコード。
 * 形式: "TS1." + base64url(deflate-raw(JSON {t, s}))
 */

const PREFIX = "TS1.";

export type Signal = { type: "offer" | "answer"; sdp: string };

/** データチャネルだけの接続に不要な行を落として短くする */
export function slimSdp(sdp: string): string {
  const drop = [/^a=extmap-allow-mixed/, /^a=msid-semantic/, /^a=max-message-size/];
  return sdp
    .split(/\r?\n/)
    .filter((l) => l && !drop.some((re) => re.test(l)))
    .join("\r\n")
    .concat("\r\n");
}

async function pipe(data: Uint8Array, stream: CompressionStream | DecompressionStream): Promise<Uint8Array> {
  const out = new Blob([data as BlobPart]).stream().pipeThrough(stream);
  return new Uint8Array(await new Response(out).arrayBuffer());
}

function toBase64Url(bytes: Uint8Array): string {
  let bin = "";
  for (const b of bytes) bin += String.fromCharCode(b);
  return btoa(bin).replace(/%/g, "-").replace(/_/g, "_").replace(/=+$/, "");
}

function fromBase64Url(s: string): Uint8Array {
  const b64 = s.replace(/-/g, "+").replace(/_/g, "/");
  const bin = atob(b64 + "=".repeat((4 - (b64.length % 4)) % 4));
  return Uint8Array.from(bin, (c) => c.charCodeAt(0));
}

export async function encodeSignal(sig: Signal): Promise<string> {
  const json = JSON.stringify({ t: sig.type === "offer" ? "o" : "a", s: slimSdp(sig.sdp) });
  const packed = await pipe(new TextEncoder().encode(json), new CompressionStream("deflate-raw"));
  return PREFIX + toBase64Url(packed);
}

export class SignalFormatError extends Error {}

export async function decodeSignal(code: string): Promise<Signal> {
  const s = code.trim();
  if (!s.startsWith(PREFIX)) throw new SignalFormatError("接続コードの形式が違います");
  try {
    const bytes = await pipe(fromBase64Url(s.slice(PREFIX.length)), new DecompressionStream("deflate-raw"));
    const { t, s: sdp } = JSON.parse(new TextDecoder().decode(bytes)) as { t: string; s: string };
    if ((t !== "o" && t !== "a") || typeof sdp !== "string" || !sdp.startsWith("v=0")) throw new Error();
    return { type: t === "o" ? "offer" : "answer", sdp };
  } catch {
    throw new SignalFormatError("接続コードを読み取れませんでした");
  }
}
```

付録B src/lib/p2p/peer.ts

```
import { type Signal, decodeSignal, encodeSignal } from "../signal";

/**
 * スマホ2台を WebRTC DataChannel で直接つなぐ。
```

```

* シグナリングサーバーは使わず、接続コード（QR）を2回やりとりする：
*   ホスト：createOffer → 相手に見せる → 相手の答えを acceptAnswer
*   ゲスト：相手のコードで answerOffer → 自分の答えを相手に見せる
* STUN はインターネット経由（別々の回線）でもつながるようにするため。同じWi-Fiなら不要。
*/
export const ICE_SERVERS: RTCIceServer[] = [{ urls: "stun:stun.l.google.com:19302" }];
const GATHER_TIMEOUT_MS = 4000;

export type RouteKind = "LAN" | "INTERNET" | "RELAY" | "UNKNOWN";

export type PeerEvents = {
  onOpen: () => void;
  onMessage: (data: unknown) => void;
  onClose: () => void;
};

/** 候補を集め終わるまで待つ（コードを1回で渡すため、trickle しない） */
function waitGathering(pc: RTCPeerConnection): Promise<void> {
  if (pc.iceGatheringState === "complete") return Promise.resolve();
  return new Promise((resolve) => {
    const done = () => {
      clearTimeout(timer);
      pc.removeEventListener("icegatheringstatechange", check);
      resolve();
    };
    const check = () => pc.iceGatheringState === "complete" && done();
    const timer = setTimeout(done, GATHER_TIMEOUT_MS);
    pc.addEventListener("icegatheringstatechange", check);
  });
}

export class Peer {
  readonly pc: RTCPeerConnection;
  private channel: RTCDataChannel | null = null;

  constructor(private events: PeerEvents, iceServers: RTCIceServer[] = ICE_SERVERS) {
    this.pc = new RTCPeerConnection({ iceServers });
    this.pc.addEventListener("datachannel", (e) => this.bind(e.channel));
    this.pc.addEventListener("connectionstatechange", () => {
      if (this.pc.connectionState === "failed" || this.pc.connectionState === "closed") this.events.onClose();
    });
  }

  private bind(ch: RTCDataChannel) {
    this.channel = ch;
    ch.addEventListener("open", () => this.events.onOpen());
    ch.addEventListener("close", () => this.events.onClose());
    ch.addEventListener("message", (e) => {
      try {
        this.events.onMessage(JSON.parse(e.data as string));
      } catch {
        // 壊れたメッセージは無視
      }
    });
  }

  private async localCode(): Promise<string> {
    await waitGathering(this.pc);
    const d = this.pc.localDescription!;
    return encodeSignal({ type: d.type as Signal["type"], sdp: d.sdp });
  }

  /** ホスト：最初の接続コードを作る */
  async createOffer(): Promise<string> {
    this.bind(this.pc.createDataChannel("game", { ordered: true }));
    await this.pc.setLocalDescription(await this.pc.createOffer());
    return this.localCode();
  }

  /** ゲスト：ホストのコードから返事のコードを作る */

```

```

async answerOffer(code: string): Promise<string> {
  const sig = await decodeSignal(code);
  if (sig.type !== "offer") throw new Error("ホストの接続コードを読み取ってね");
  await this.pc.setRemoteDescription(sig);
  await this.pc.setLocalDescription(await this.pc.createAnswer());
  return this.localCode();
}

/** ホスト: ゲストの返事を受け取ってつなぐ */
async acceptAnswer(code: string): Promise<void> {
  const sig = await decodeSignal(code);
  if (sig.type !== "answer") throw new Error("ゲストの返事のコードを読み取ってね");
  await this.pc.setRemoteDescription(sig);
}

send(data: unknown): boolean {
  if (this.channel?.readyState !== "open") return false;
  this.channel.send(JSON.stringify(data));
  return true;
}

/** 実際に使われている経路 (同じネットワーク内 / インターネット経由 / 中継) */
async route(): Promise<RouteKind> {
  const stats = await this.pc.getStats();
  let pairId: string | undefined;
  stats.forEach((s) => {
    if (s.type === "transport" && s.selectedCandidatePairId) pairId = s.selectedCandidatePairId;
  });
  let pair: RTCIceCandidatePairStats | undefined;
  stats.forEach((s) => {
    if (s.type === "candidate-pair" && (s.id === pairId || (!pairId && s.nominated && s.state === "succeeded")))) pair = s;
  });
  if (!pair) return "UNKNOWN";
  const local = stats.get(pair.localCandidateId) as { candidateType?: string } | undefined;
  const remote = stats.get(pair.remoteCandidateId) as { candidateType?: string } | undefined;
  const types = [local?.candidateType, remote?.candidateType];
  if (types.includes("relay")) return "RELAY";
  // 両端がローカルのアドレスなら同じネットワーク内、どちらかがルーター越し (srflx) ならインターネット経由
  return types.every((t) => t === "host" || t === "prflx") ? "LAN" : "INTERNET";
}

close() {
  this.channel?.close();
  this.pc.close();
}
}

```

付録C React 用 QR 部品

src/components/QrCode.tsx

```

"use client";

import QRCode from "qrcode";
import { useEffect, useRef, useState } from "react";

/** 接続コードを QR で表示する (白地・余白付きで読み取りやすく) */
export function QrCode({ text }: { text: string }) {
  const canvas = useRef<HTMLCanvasElement>(null);
  // 細かいQRなので、カメラで読みやすいよう画面幅いっぱいまで大きくする
  const [size, setSize] = useState(280);
  useEffect(() => setSize(Math.min(window.innerWidth, 480) - 32), []);
  useEffect(() => {
    if (!canvas.current) return;
    QRCode.toCanvas(canvas.current, text, { errorCorrectionLevel: "L", margin: 2, width: size }).catch(() => {});
  }, [text, size]);
}

```

```

return <canvas ref={canvas} data-qr className="mx-auto rounded-xl bg-white" style={{ width: size, height: size }} />;
}

```

src/components/QrScanner.tsx

```

"use client";

import jsQR from "jsqr";
import { useEffect, useRef, useState } from "react";

const SCAN_INTERVAL_MS = 150;

/** カメラで QR を読み取る。読めたら onResult を1回だけ呼ぶ */
export function QrScanner({ onResult }: { onResult: (text: string) => void }) {
  const video = useRef<HTMLVideoElement>(null);
  const [error, setError] = useState<string | null>(null);
  const done = useRef(false);

  useEffect(() => {
    let stream: MediaStream | null = null;
    let timer: ReturnType<typeof setInterval> | null = null;
    const canvas = document.createElement("canvas");
    const ctx = canvas.getContext("2d", { willReadFrequently: true });

    (async () => {
      try {
        stream = await navigator.mediaDevices.getUserMedia({
          video: { facingMode: "environment", width: { ideal: 1280 }, height: { ideal: 720 } },
          audio: false,
        });
      } catch {
        setError("カメラを使えませんでした。ブラウザのカメラ許可を確認するか、下のコード入力を使ってね");
        return;
      }
      const v = video.current;
      if (!v) return;
      v.srcObject = stream;
      await v.play().catch(() => {});
      timer = setInterval(() => {
        if (done.current || !ctx || v.videoWidth === 0) return;
        canvas.width = v.videoWidth;
        canvas.height = v.videoHeight;
        ctx.drawImage(v, 0, 0);
        const img = ctx.getImageData(0, 0, canvas.width, canvas.height);
        const code = jsQR(img.data, img.width, img.height, { inversionAttempts: "dontInvert" });
        if (code?.data) {
          done.current = true;
          onResult(code.data);
        }
      }, SCAN_INTERVAL_MS);
    })();

    return () => {
      if (timer) clearInterval(timer);
      stream?.getTracks().forEach((t) => t.stop());
    };
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

  if (error) return <p className="rounded-xl bg-bg p-4 text-note text-[#FF8A8D]">{error}</p>;
  return (
    <div className="relative mx-auto aspect-square w-full max-w-[320px] overflow-hidden rounded-2xl bg-black">
      <video ref={video} playsInline muted className="h-full w-full object-cover" />
      <div className="pointer-events-none absolute inset-[15%] rounded-xl border-2 border-gold/80" />
    </div>
  );
}

```

※ Tailwind のクラス名 (className) は見た目のためだけ。他の CSS に置き換えてよい。

付録D .github/workflows/deploy-pages.yml

```
name: Deploy to GitHub Pages

on:
  push:
    branches: [ "claude/pdf-review-before-dev-jwyd3k", "main" ]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: true

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm test
      - name: Build (static export)
        run: npm run build
      - env:
          # https://<owner>.github.io/<repo>/ で配信するためのサブパス
          NEXT_PUBLIC_BASE_PATH: /${{ github.event.repository.name }}
        uses: actions/upload-pages-artifact@v3
        with:
          path: out

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - id: deployment
        uses: actions/deploy-pages@v4
```

Vite 等の場合: npm run build の出力先に合わせて path: dist に変更し、ベースパスは vite build --base=/\${{ github.event.repository.name }}/ など渡す。branches は公開したいブランチに合わせる。